Introduction to C Programming

1. The History of C and C++

The Origins of C

C was developed in the early 1970s by Dennis Ritchie at Bell Labs. It was derived from the B language and combined features of assembly language and high-level languages. C became widely adopted due to its efficiency, simplicity, and portability.

The Rise of C++

C++ was introduced by Bjarne Stroustrup in 1983 as an extension of C, adding object-oriented programming (OOP) features such as classes and inheritance. It allowed developers to write code that was both efficient and reusable, making it a preferred choice for large-scale software projects.

Why We Still Use C and C++ Today

1. **Performance**: Both languages provide low-level control over hardware and memory, making them ideal for system-level programming.
2. **Portability**: Programs written in C/C++ can be compiled and run on different platforms with minimal modifications.
3. **Legacy Code**: Many existing systems are built using C/C++, and maintaining them requires knowledge of these languages.
4. **Versatility**: They are used in various domains, including operating systems, embedded systems, game development, and scientific computing.

---

2. The Basics of C Programming

The `#include` Directive

The `#include` directive is used to include external libraries in your program. For example:

```
# include <stdio.h>
```

This includes the **Standard Input/Output** library, which provides functions like `printf` and `scanf`.

Example

```
# include <stdio.h>
```

```
int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Output:

```
Hello, World!
```

Comments in C

- **Single-line Comment**: `// This is a comment`
- **Multi-line Comment**: `/* This is a multi-line comment */`

---

3. Data Types in C

Primitive Data Types

1. **Integer (`int`)**:
    - Used for whole numbers.
    - Example: `int a = 10;`
2. **Floating-Point (`float`)**:
    - Used for decimal numbers.
    - Example: `float b = 3.14;`
3. **Double (`double`)**:
    - Used for higher precision decimal numbers.
    - Example: `double c = 3.141592653589;`
4. **Character (`char`)**:
    - Used for single characters.
    - Example: `char d = 'A';`
5. **String**:
    - Not a primitive data type but represented as an array of `char`.
    - Example: `char str[] = "Hello";`

Examples

```
# include <stdio.h>

int main() {
    int a = 10;
```

```
    float b = 3.14;
    double c = 3.14159;
    char d = 'A';
    char str[] = "Hello, World!";

    printf("Integer: %d\n", a);
    printf("Float: %.2f\n", b);
    printf("Double: %.5f\n", c);
    printf("Character: %c\n", d);
    printf("String: %s\n", str);

    return 0;
}
```

Output:

```
Integer: 10
Float: 3.14
Double: 3.14159
Character: A
String: Hello, World!
```

---

4. Functions in C

What is a Function?

A function is a reusable block of code that performs a specific task. Functions help make programs modular and easier to understand.

Defining a Function

```
return_type function_name(parameters) {
    // Code block
    return value; // Optional
}
```

Example: Function to Add Two Numbers

```
# include <stdio.h>

int add(int x, int y) {
    return x + y;
}
```

```
int main() {
    int result = add(5, 7);
    printf("Sum: %d\n", result);
    return 0;
}
```

Output:

```
Sum: 12
```

Types of Functions

1. **No Return, No Parameters**:

   ```
   void greet() {
       printf("Hello!\n");
   }
   ```

2. **With Return, No Parameters**:

   ```
   int getValue() {
       return 42;
   }
   ```

3. **With Return and Parameters**:

   ```
   int multiply(int a, int b) {
       return a * b;
   }
   ```

---

5. Control Structures and Loops

For Loop

Used for iterating a fixed number of times.

```
# include <stdio.h>

int main() {
    for (int i = 0; i < 5; i++) {
        printf("i = %d\n", i);
    }
```

```
    return 0;
}
```

While Loop

Used when the number of iterations is not fixed.

```
# include <stdio.h>

int main() {
    int i = 0;
    while (i < 5) {
        printf("i = %d\n", i);
        i++;
    }
    return 0;
}
```

Do-While Loop

Executes the loop body at least once.

```
# include <stdio.h>

int main() {
    int i = 0;
    do {
        printf("i = %d\n", i);
        i++;
    } while (i < 5);
    return 0;
}
```

---

6. Basic Programs in C

Program to Find the Factorial of a Number

```
# include <stdio.h>

int factorial(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
```

```
    return result;
}

int main() {
    int num = 5;
    printf("Factorial of %d is %d\n", num, factorial(num));
    return 0;
}
```

Program to Check if a Number is Prime

```
# include <stdio.h>
# include <stdbool.h>
# include <math.h> // Include for sqrt()

bool isPrime(int n) {
    if (n <= 1) return false;
    // Optimization: Check divisibility only up to sqrt(n)
    for (int i = 2; i <= sqrt(n); i++) {
        if (n % i == 0) return false;
    }
    return true;
}

int main() {
    int num = 7;
    if (isPrime(num)) {
        printf("%d is a prime number.\n", num);
    } else {
        printf("%d is not a prime number.\n", num);
    }

    num = 12;
    if (isPrime(num)) {
        printf("%d is a prime number.\n", num);
    } else {
        printf("%d is not a prime number.\n", num);
    }
    return 0;
}
```

END of Introduction

Numerical Computation in C: Fundamentals

Introduction

Numerical computation is the cornerstone of many scientific and engineering disciplines. In this tutorial, we will cover fundamental concepts such as binary numbers, computer representation of real numbers, machine precision, and rounding errors. Through practical examples and implementations in C, we will explore these topics in depth to build a strong foundation for numerical computation.

---

1. Binary Numbers

Definition and Explanation

The binary number system, or base-2, is the foundation of computer arithmetic. Unlike the decimal system (base-10), it uses only two digits: 0 and 1. Computers rely on binary to represent data and perform arithmetic operations efficiently.

Conversion

Decimal to Binary

To convert a decimal integer to binary:

1. Divide the number by 2.
2. Record the remainder (0 or 1).
3. Continue dividing the quotient by 2 until it reaches 0.
4. Write the remainders in reverse order.

Example: Convert 53 (base-10) to binary:

```
53 / 2 = 26 R1
26 / 2 = 13 R0
13 / 2 =  6 R1
 6 / 2 =  3 R0
 3 / 2 =  1 R1
 1 / 2 =  0 R1
```

Result: **53 = 110101 (base-2)**

Binary to Decimal

To convert a binary number to decimal:

1. Multiply each binary digit by 2 raised to its position index (starting from 0 on the right).
2. Sum all the results.

Example: Convert 110101 (base-2) to decimal:

```
= 1 * 2^5 + 1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0
= 32 + 16 + 0 + 4 + 0 + 1
= 53 (base-10)
```

Practice Problems

1. Convert the following decimal numbers to binary:
    - 64
    - 17
    - 79
2. Convert the following binary numbers to decimal:
    - 1010101
    - 110.10

Solutions in C

Decimal to Binary Conversion

```c
# include <stdio.h>

void decimalToBinary(int n) {
    int binary[32];
    int i = 0;
    while (n > 0) {
        binary[i] = n % 2;
        n = n / 2;
        i++;
    }
    for (int j = i - 1; j >= 0; j--) {
        printf("%d", binary[j]);
    }
    printf("\n");
}

int main() {
    int number = 53;
    printf("Binary of %d: ", number);
    decimalToBinary(number);
    return 0;
}
```

Binary to Decimal Conversion

```
# include <stdio.h>
# include <math.h>

int binaryToDecimal(const char *binary) {
    int decimal = 0;
    int len = strlen(binary);
    for (int i = 0; i < len; i++) {
        if (binary[i] == '1') {
            decimal += pow(2, len - 1 - i);
        }
    }
    return decimal;
}

int main() {
    const char *binary = "110101";
    printf("Decimal of %s: %d\n", binary, binaryToDecimal(binary));
    return 0;
}
```

Next week: Pointers and File handling.

```
Start coding or generate with AI.
```